# Assignment 6:

Name: Leon Lobo

Admission No: U24AI043

**Q1 Use the city map given in Q1 of assignment 1 and implement the greedy best first search and A* algorithm to reach Boston city from Chicago and compare your results in terms of the number of cities explored before reaching your destination. Consider the heuristic values as given below,**

**City h(n) to Boston (miles)**

**Boston 0**
**Providence 50**
**Portland 107**
**New York 215**
**Philadelphia 270**
**Baltimore 360**
**Syracuse 260**
**Buffalo 400**
**Pittsburgh 470**
**Cleveland 550**
**Columbus 640**
**Detroit 610**
**Indianapolis 780**
**Chicago 860**

Code:

Lab 6 > A6_Q1.py > main

```python
cities=[
    "Syracuse", "Buffalo", "Detroit", "Cleveland", "Pittsburgh",
    "Columbus", "Indianapolis", "New York", "Philadelphia",
    "Baltimore", "Providence", "Boston", "Portland", "Chicago"
]


graph=[
#  Sy  Bu  De  Cl  Pi  Co  In  NY  Ph  Ba  Pr  Bo  Po  Ch
 [ 0, 150, 0, 0, 0, 0, 0, 254, 253, 0, 0, 312, 0, 0],  # Syracuse
 [150, 0, 256, 189, 215, 0, 0, 0, 0, 0, 0, 0, 0, 0],  # Buffalo
 [ 0, 256, 0, 169, 0, 0, 0, 0, 0, 0, 0, 0, 0, 283],  # Detroit
 [ 0, 189, 169, 0, 134, 144, 0, 0, 0, 0, 0, 0, 0, 345],  # Cleveland
 [ 0, 215, 0, 134, 0, 185, 0, 0, 305, 247, 0, 0, 0, 0],  # Pittsburgh
 [ 0, 0, 0, 144, 185, 0, 176, 0, 0, 0, 0, 0, 0, 0],  # Columbus
 [ 0, 0, 0, 0, 0, 176, 0, 0, 0, 0, 0, 0, 0, 182],  # Indianapolis
 [254, 0, 0, 0, 0, 0, 0, 0, 97, 0, 181, 215, 0, 0],  # New York
 [253, 0, 0, 0, 305, 0, 0, 97, 0, 101, 0, 0, 0, 0],  # Philadelphia
 [ 0, 0, 0, 0, 247, 0, 0, 0, 101, 0, 0, 0, 0, 0],  # Baltimore
 [ 0, 0, 0, 0, 0, 0, 0, 181, 0, 0, 0, 50, 0, 0],  # Providence
 [312, 0, 0, 0, 0, 0, 0, 215, 0, 0, 50, 0, 107, 0],  # Boston
 [ 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 107, 0, 0],  # Portland
 [ 0, 0, 283, 345, 0, 0, 182, 0, 0, 0, 0, 0, 0, 0]   # Chicago
]


class Node:
    def __init__(self, state, parent=None, cost=0):
        self.state=state
        self.parent=parent
        self.cost=cost

def h(node):
    heuristic=(0, 50, 107, 215, 270, 360, 260, 400, 470, 550, 640, 610, 780, 860)
    return heuristic[node.state]


def g(node):
    return node.cost


def f(node):
    return h(node)+g(node)


class pq:
    def __init__(self):
        self.queue=[]

    def push(self, node):
        self.queue.append(node)

    def pop(self,f):
        best=0
        for i in range(1, len(self.queue)):
```

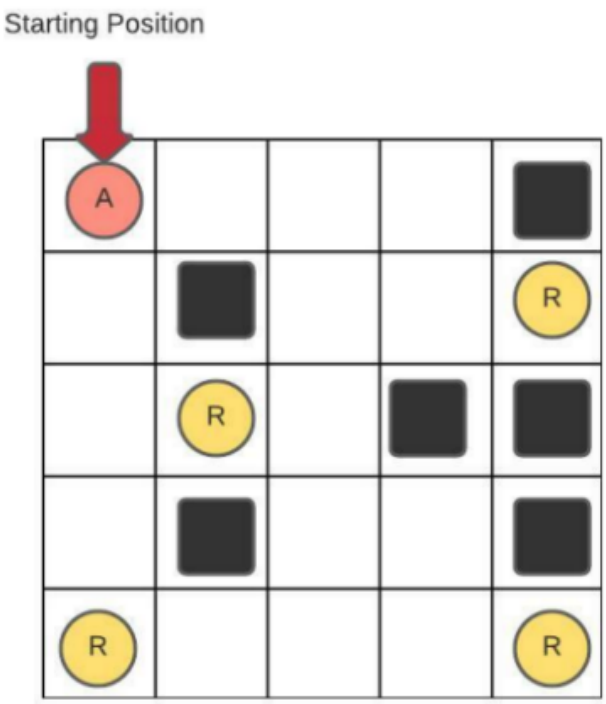
```python
class pq:
    def pop(self,f):
            if f(self.queue[i])<f(self.queue[best]):
                best=i
        return self.queue.pop(best)

    def is_empty(self):
        return len(self.queue)==0


def expand(node):
    children=[]
    for j in range(len(cities)):
        if graph[node.state][j]!=0:
            child=Node(
                j,
                node,
                node.cost+graph[node.state][j]
            )
            children.append(child)
    return children


def best_first_search(start, dest, f):
    start_node=Node(start)
    frontier=pq()
    frontier.push(start_node)
    reached=[None]*len(cities)
    reached[start]=start_node
    global count_node
    count_node=0

    while not frontier.is_empty():
        node=frontier.pop(f)
        count_node+=1
        print(f"({cities[node.state]}, {node.cost})",end="")

        if node.state==dest:
            return node

        print("->",end="")
        for child in expand(node):
            s=child.state
            if reached[s] is None or child.cost<reached[s].cost:
                reached[s]=child
                frontier.push(child)

    return None


def greedy_bfs(start, dest):
    return best_first_search(start, dest, h)

def A_star(start, dest):
    return best_first_search(start, dest, f)
```

```python
def A_star(start, dest):
    return best_first_search(start, dest, f)


def print_path(node):
    path=[]
    while node:
        path.append(cities[node.state])
        node=node.parent
    path.reverse()
    print(" → ".join(path))


def main():
    start=cities.index("Chicago")
    dest=cities.index("Boston")
    print("Greedy Best First Search:-")
    result=greedy_bfs(start, dest)

    if result:
        print("\nPath found:")
        print_path(result)
        print("Minimmum path cost:", result.cost)
        print("Nodes explored:",count_node)
    else:
        print("No path found")

    print("A* search:-")
    result=A_star(start, dest)

    if result:
        print("\nPath found:")
        print_path(result)
        print("Minimmum path cost:", result.cost)
        print("Nodes explored:",count_node)
    else:
        print("No path found")



if __name__=='__main__':
    main()
```

Output:

```
Greedy Best First Search:-
(Chicago, 0)->(Detroit, 283)->(Buffalo, 539)->(Syracuse, 689)->(Cleveland, 345)->(Buffalo, 534)->(Syracuse, 684)->(Indianapolis, 182)
->(Pittsburgh, 754)->(Pittsburgh, 479)->(Columbus, 489)->(Columbus, 358)->(New York, 943)->(New York, 938)->(Philadelphia, 942)->(Phi
ladelphia, 937)->(Philadelphia, 784)->(New York, 881)->(Baltimore, 1001)->(Baltimore, 726)->(Boston, 1001)
Path found:
Chicago → Detroit → Buffalo → Syracuse → Boston
Minimmum path cost: 1001
Nodes explored: 21
A* search:-
(Chicago, 0)->(Detroit, 283)->(Indianapolis, 182)->(Cleveland, 345)->(Buffalo, 534)->(Buffalo, 539)->(Syracuse, 684)->(Columbus, 358)
->(Pittsburgh, 479)->(Philadelphia, 784)->(Baltimore, 726)->(New York, 881)->(New York, 938)->(Philadelphia, 937)->(Boston, 996)
Path found:
Chicago → Cleveland → Buffalo → Syracuse → Boston
Minimmum path cost: 996
Nodes explored: 15
```

**Q2 Consider a maze as a 5x5 matrix input where 0 denotes an empty tile, 1 denotes an obstruction/wall, 2 denotes the start state and 3 denotes the reward. Assume valid actions as L,R,U,D where L=move_left, R=move_right, U=move_up, D=move_down,. Use the A* algorithm on the resultant maze for your agent to reach all the rewards, and keep a record of the tiles visited on the way. Consider the rewards as goals. If all goals are completed, then one can come out of the maze. Assume heuristic cost and evaluation cost with proper justification.**



# Code:

Lab 6 > A6_Q2.py > A_star

```python
matrix=[
    [2, 0, 0, 0, 1],
    [0, 1, 0, 0, 3],
    [0, 3, 0, 1, 1],
    [0, 1, 0, 0, 1],
    [3, 0, 0, 0, 3]
]

class Node:
    def __init__(self, state, action, parent=None, cost=0, goals=[1, 1, 1, 1]):
        self.state=state
        self.parent=parent
        self.action=action
        self.cost=cost
        self.goals=goals


def h(node):
    x=node.state[0]
    y=node.state[1]
    heuristic=[]
    heuristic.append((abs(x-1)+abs(y-4))*node.goals[0])
    heuristic.append((abs(x-2)+abs(y-1))*node.goals[1])
    heuristic.append((abs(x-4)+abs(y-0))*node.goals[2])
    heuristic.append((abs(x-4)+abs(y-4))*node.goals[3])
    min_h=float('inf')
    for h in heuristic:
        if (h==0 and node.goals[heuristic.index(h)]==1) or h!=0:
            min_h=min(h,min_h)
    return min_h

def g(node):
    return node.cost


def f(node):
    cost=g(node)
```

Lab 6 > A6_Q2.py > A_star

```python
def f(node):
    heuristic=h(node)
    return (cost+heuristic)



class pq:
    def __init__(self):
        self.queue=[]

    def push(self, node):
        self.queue.append(node)

    def pop(self):
        best=0
        for i in range(1, len(self.queue)):
            if f(self.queue[i])<f(self.queue[best]):
                best=i
        return self.queue.pop(best)

    def is_empty(self):
        return len(self.queue)==0


def expand(problem, node):
    s=node.state
    children=[]

    moves=[(-1, 0), (1, 0), (0, -1), (0, 1)]

    for dx, dy in moves:
        s0=(s[0]+dx, s[1]+dy)

        if 0<=s0[0]<len(problem) and 0<=s0[1]<len(problem[0]):

            if problem[s0[0]][s0[1]]!=1:
                cost=node.cost+1
```

Lab 6 > A6_Q2.py > A_star

```python
def expand(problem, node):
                child=Node(s0, (dx,dy), node, cost, node.goals.copy())
                children.append(child)

    return children


def A_star(problem, start, rewards):

    node=Node(start, None, None, 0)

    frontier=pq()
    frontier.push(node)
    reached={}
    reached[(start, tuple(node.goals))]=0
    explored_count=0

    while frontier.queue:

        node=frontier.pop()
        explored_count+=1

        if node.state in rewards:
            node.goals[rewards.index(node.state)]=0

        if node.parent is not None:
            print(node.parent.state, "->", node.state, ",",g(node), h(node), f(node), node.goals)

        if all(g==0 for g in node.goals):
            return node, explored_count

        for child in expand(problem, node):
            s=(child.state,tuple(child.goals))
            if s not in reached or child.cost<reached[s]:
                reached[s]=child.cost
                frontier.push(child)
```

```python
            s=(child.state,tuple(child.goals))
            if s not in reached or child.cost<reached[s]:
                reached[s]=child.cost
                frontier.push(child)

    return None, explored_count

def print_path(node):
    path=[]
    while node:
        path.append(f"({node.state[0]},{node.state[1]})")
        node=node.parent
    path.reverse()
    print(" → ".join(path))

def main():
    goals=[(1, 4), (2, 1), (4, 0), (4, 4)]
    ans,count=A_star(matrix, (0, 0), goals)
    if ans:
        print("\nPath to collect all rewards:")
        print_path(ans)
    print("Nodes explored:", count)

if __name__=="__main__":
    main()
```

Output:

```
PS C:\Education\SVNIT\Coding\AI> python
(0, 0) -> (1, 0) , 1 2 3 [1, 1, 1, 1]
(0, 0) -> (0, 1) , 1 2 3 [1, 1, 1, 1]
(1, 0) -> (2, 0) , 2 1 3 [1, 1, 1, 1]
(2, 0) -> (2, 1) , 3 3 6 [1, 0, 1, 1]
(2, 0) -> (3, 0) , 3 1 4 [1, 1, 1, 1]
(3, 0) -> (4, 0) , 4 3 7 [1, 1, 0, 1]
(0, 1) -> (0, 2) , 2 3 5 [1, 1, 1, 1]
(0, 2) -> (1, 2) , 3 2 5 [1, 1, 1, 1]
(0, 2) -> (0, 3) , 3 2 5 [1, 1, 1, 1]
(1, 2) -> (2, 2) , 4 1 5 [1, 1, 1, 1]
(1, 2) -> (1, 3) , 4 1 5 [1, 1, 1, 1]
(1, 3) -> (1, 4) , 5 3 8 [0, 1, 1, 1]
(2, 1) -> (2, 0) , 4 2 6 [1, 0, 1, 1]
(2, 0) -> (3, 0) , 5 1 6 [1, 0, 1, 1]
(2, 1) -> (2, 2) , 4 3 7 [1, 0, 1, 1]
(4, 0) -> (3, 0) , 5 2 7 [1, 1, 0, 1]
(4, 0) -> (4, 1) , 5 2 7 [1, 1, 0, 1]
(2, 2) -> (3, 2) , 5 2 7 [1, 1, 1, 1]
(2, 2) -> (1, 2) , 5 2 7 [1, 0, 1, 1]
(3, 0) -> (2, 0) , 6 1 7 [1, 1, 0, 1]
(1, 2) -> (1, 3) , 6 1 7 [1, 0, 1, 1]
(2, 0) -> (2, 1) , 7 4 11 [1, 0, 0, 1]
(1, 3) -> (1, 4) , 7 3 10 [0, 0, 1, 1]
(2, 0) -> (1, 0) , 5 3 8 [1, 0, 1, 1]
(2, 0) -> (2, 1) , 5 3 8 [1, 0, 1, 1]
(2, 2) -> (3, 2) , 5 3 8 [1, 0, 1, 1]
(4, 1) -> (4, 2) , 6 2 8 [1, 1, 0, 1]
(3, 2) -> (4, 2) , 6 2 8 [1, 1, 1, 1]
(3, 2) -> (3, 3) , 6 2 8 [1, 1, 1, 1]
(3, 2) -> (4, 2) , 6 2 8 [1, 0, 1, 1]
(3, 2) -> (3, 3) , 6 2 8 [1, 0, 1, 1]
(4, 2) -> (4, 3) , 7 1 8 [1, 1, 0, 1]
(4, 2) -> (4, 1) , 7 1 8 [1, 1, 1, 1]
(4, 2) -> (4, 3) , 7 1 8 [1, 1, 1, 1]
(4, 2) -> (4, 1) , 7 1 8 [1, 0, 1, 1]
(4, 2) -> (4, 3) , 7 1 8 [1, 0, 1, 1]
(4, 3) -> (4, 4) , 8 3 11 [1, 1, 1, 0]
(1, 4) -> (1, 3) , 6 3 9 [0, 1, 1, 1]
(3, 0) -> (4, 0) , 6 3 9 [1, 1, 0, 1]
(1, 2) -> (0, 2) , 6 3 9 [1, 0, 1, 1]
(2, 0) -> (1, 0) , 7 2 9 [1, 1, 0, 1]
(1, 3) -> (0, 3) , 7 2 9 [1, 0, 1, 1]
(4, 2) -> (3, 2) , 7 2 9 [1, 1, 0, 1]
(1, 3) -> (1, 2) , 7 2 9 [0, 1, 1, 1]
```

```
(3, 2) -> (2, 2) , 8 1 9 [1, 1, 0, 1]
(1, 2) -> (2, 2) , 8 1 9 [0, 1, 1, 1]
(3, 0) -> (4, 0) , 6 4 10 [1, 0, 0, 1]
(1, 0) -> (0, 0) , 6 4 10 [1, 0, 1, 1]
(4, 3) -> (3, 3) , 8 2 10 [1, 1, 0, 1]
(1, 3) -> (1, 4) , 7 3 10 [0, 1, 1, 1]
(4, 0) -> (4, 1) , 7 3 10 [1, 0, 0, 1]
(4, 1) -> (4, 2) , 8 2 10 [1, 0, 0, 1]
(4, 2) -> (4, 3) , 9 1 10 [1, 0, 0, 1]
(2, 1) -> (2, 2) , 8 3 11 [1, 0, 0, 1]
(4, 3) -> (4, 4) , 8 3 11 [1, 1, 0, 0]
(4, 3) -> (4, 4) , 8 3 11 [1, 0, 1, 0]
(1, 3) -> (0, 3) , 7 4 11 [0, 1, 1, 1]
(0, 2) -> (0, 1) , 7 4 11 [1, 0, 1, 1]
(1, 0) -> (0, 0) , 8 3 11 [1, 1, 0, 1]
(1, 2) -> (0, 2) , 8 3 11 [0, 1, 1, 1]
(2, 2) -> (1, 2) , 9 2 11 [1, 1, 0, 1]
(2, 2) -> (3, 2) , 9 2 11 [0, 1, 1, 1]
(2, 2) -> (1, 2) , 9 2 11 [1, 0, 0, 1]
(0, 0) -> (0, 1) , 9 2 11 [1, 1, 0, 1]
(0, 2) -> (0, 1) , 9 2 11 [0, 1, 1, 1]
(1, 2) -> (1, 3) , 10 1 11 [1, 1, 0, 1]
(1, 2) -> (1, 3) , 10 1 11 [1, 0, 0, 1]
(1, 3) -> (1, 4) , 11 3 14 [0, 1, 0, 1]
(1, 3) -> (1, 4) , 11 3 14 [0, 0, 0, 1]
(1, 4) -> (1, 3) , 8 4 12 [0, 0, 1, 1]
(4, 4) -> (4, 3) , 9 3 12 [1, 1, 1, 0]
(2, 2) -> (2, 1) , 9 3 12 [0, 0, 1, 1]
(4, 0) -> (3, 0) , 7 5 12 [1, 0, 0, 1]
(4, 1) -> (4, 0) , 8 4 12 [1, 0, 0, 1]
(4, 2) -> (3, 2) , 9 3 12 [1, 0, 0, 1]
(4, 3) -> (3, 3) , 10 2 12 [1, 0, 0, 1]
(4, 4) -> (4, 3) , 9 3 12 [1, 0, 1, 0]
(3, 2) -> (4, 2) , 10 2 12 [0, 1, 1, 1]
(3, 2) -> (3, 3) , 10 2 12 [0, 1, 1, 1]
(1, 3) -> (1, 4) , 9 3 12 [0, 0, 1, 1]
(4, 3) -> (4, 2) , 10 2 12 [1, 1, 1, 0]
(2, 1) -> (2, 0) , 10 2 12 [0, 0, 1, 1]
(4, 3) -> (4, 2) , 10 2 12 [1, 0, 1, 0]
(4, 2) -> (4, 1) , 11 1 12 [0, 1, 1, 1]
(4, 2) -> (4, 3) , 11 1 12 [0, 1, 1, 1]
(4, 2) -> (4, 1) , 11 1 12 [1, 1, 1, 0]
(2, 0) -> (3, 0) , 11 1 12 [0, 0, 1, 1]
(4, 2) -> (4, 1) , 11 1 12 [1, 0, 1, 0]
(4, 1) -> (4, 0) , 12 3 15 [1, 1, 0, 0]
```

```
(2, 1) -> (2, 0) , 8 5 13 [1, 0, 0, 1]
(4, 3) -> (4, 4) , 10 3 13 [1, 0, 0, 0]
(2, 2) -> (2, 1) , 9 4 13 [1, 0, 0, 1]
(4, 4) -> (4, 3) , 9 4 13 [1, 1, 0, 0]
(1, 2) -> (0, 2) , 10 3 13 [1, 1, 0, 1]
(1, 2) -> (0, 2) , 10 3 13 [1, 0, 0, 1]
(0, 1) -> (0, 0) , 10 3 13 [0, 1, 1, 1]
(1, 3) -> (0, 3) , 11 2 13 [1, 1, 0, 1]
(1, 3) -> (0, 3) , 11 2 13 [1, 0, 0, 1]
(4, 3) -> (3, 3) , 10 3 13 [1, 1, 1, 0]
(4, 3) -> (4, 4) , 10 3 13 [1, 1, 1, 0]
(4, 3) -> (3, 3) , 10 3 13 [1, 0, 1, 0]
(4, 3) -> (4, 4) , 10 3 13 [1, 0, 1, 0]
(4, 2) -> (3, 2) , 11 2 13 [1, 1, 1, 0]
(2, 0) -> (1, 0) , 9 4 13 [1, 0, 0, 1]
(4, 3) -> (3, 3) , 10 3 13 [1, 1, 0, 0]
(4, 3) -> (4, 2) , 10 3 13 [1, 1, 0, 0]
(4, 3) -> (4, 4) , 10 3 13 [1, 1, 0, 0]
(0, 0) -> (1, 0) , 11 2 13 [0, 1, 1, 1]
(3, 2) -> (2, 2) , 12 1 13 [1, 1, 1, 0]
(3, 3) -> (3, 2) , 11 2 13 [1, 1, 0, 0]
(4, 2) -> (4, 1) , 11 2 13 [1, 1, 0, 0]
(1, 0) -> (2, 0) , 12 1 13 [0, 1, 1, 1]
(2, 2) -> (2, 1) , 13 3 16 [1, 0, 1, 0]
(3, 2) -> (2, 2) , 12 1 13 [1, 1, 0, 0]
(2, 2) -> (2, 1) , 13 4 17 [1, 0, 0, 0]
(1, 3) -> (0, 3) , 9 5 14 [0, 0, 1, 1]
(1, 3) -> (1, 2) , 9 5 14 [0, 0, 1, 1]
(2, 1) -> (2, 2) , 10 4 14 [0, 0, 1, 1]
(2, 0) -> (1, 0) , 11 3 14 [0, 0, 1, 1]
(2, 0) -> (2, 1) , 11 3 14 [0, 0, 1, 1]
(4, 2) -> (3, 2) , 11 3 14 [1, 0, 1, 0]
(2, 0) -> (3, 0) , 13 1 14 [0, 1, 1, 1]
(2, 2) -> (3, 2) , 11 3 14 [0, 0, 1, 1]
(3, 2) -> (4, 2) , 12 2 14 [0, 0, 1, 1]
(3, 2) -> (3, 3) , 12 2 14 [0, 0, 1, 1]
(4, 2) -> (4, 1) , 13 1 14 [0, 0, 1, 1]
(4, 2) -> (4, 3) , 13 1 14 [0, 0, 1, 1]
(1, 4) -> (1, 3) , 12 3 15 [0, 1, 0, 1]
(4, 1) -> (4, 0) , 12 3 15 [0, 1, 0, 1]
(4, 0) -> (3, 0) , 13 2 15 [1, 1, 0, 0]
(4, 0) -> (4, 1) , 13 2 15 [1, 1, 0, 0]
(4, 4) -> (4, 3) , 11 4 15 [1, 0, 0, 0]
(0, 2) -> (0, 1) , 11 4 15 [1, 0, 0, 1]
(1, 0) -> (0, 0) , 10 5 15 [1, 0, 0, 1]
```

```
(1, 4) -> (1, 3) , 12 4 16 [0, 0, 0, 1]
(4, 3) -> (4, 4) , 12 4 16 [0, 1, 1, 0]
(3, 0) -> (4, 0) , 12 4 16 [0, 0, 0, 1]
(2, 1) -> (2, 0) , 14 2 16 [1, 0, 1, 0]
(0, 3) -> (0, 2) , 10 6 16 [0, 0, 1, 1]
(1, 0) -> (0, 0) , 12 4 16 [0, 0, 1, 1]
(1, 3) -> (1, 4) , 13 3 16 [0, 1, 0, 1]
(2, 2) -> (2, 1) , 13 3 16 [1, 0, 1, 0]
(4, 1) -> (4, 2) , 14 2 16 [0, 1, 0, 1]
(1, 3) -> (1, 4) , 13 3 16 [0, 0, 0, 1]
(4, 4) -> (4, 3) , 13 3 16 [0, 1, 1, 0]
(4, 0) -> (4, 1) , 13 3 16 [0, 0, 0, 1]
(2, 0) -> (3, 0) , 15 1 16 [1, 0, 1, 0]
(0, 2) -> (0, 1) , 11 5 16 [0, 0, 1, 1]
(4, 2) -> (4, 3) , 15 1 16 [0, 1, 0, 1]
(4, 3) -> (4, 2) , 14 2 16 [0, 1, 1, 0]
(4, 1) -> (4, 2) , 14 2 16 [0, 0, 0, 1]
(4, 2) -> (4, 1) , 15 1 16 [0, 1, 1, 0]
(4, 2) -> (4, 3) , 15 1 16 [0, 0, 0, 1]
(2, 1) -> (2, 2) , 14 3 17 [1, 0, 1, 0]
(2, 1) -> (2, 2) , 14 3 17 [1, 0, 0, 0]
(1, 3) -> (0, 3) , 13 4 17 [0, 1, 0, 1]
(4, 3) -> (4, 2) , 12 5 17 [1, 0, 0, 0]
(1, 2) -> (0, 2) , 14 3 17 [1, 1, 1, 0]
(1, 2) -> (0, 2) , 14 3 17 [1, 1, 0, 0]
(1, 2) -> (0, 2) , 14 3 17 [0, 1, 0, 1]
(3, 0) -> (4, 0) , 14 3 17 [0, 1, 0, 1]
(2, 0) -> (1, 0) , 15 2 17 [1, 1, 0, 0]
(3, 3) -> (3, 2) , 13 4 17 [1, 0, 0, 0]
(1, 3) -> (0, 3) , 15 2 17 [1, 1, 1, 0]
(1, 3) -> (0, 3) , 15 2 17 [1, 1, 0, 0]
(1, 2) -> (0, 2) , 14 3 17 [1, 0, 1, 0]
(2, 2) -> (3, 2) , 15 2 17 [0, 1, 0, 1]
(2, 0) -> (1, 0) , 15 2 17 [0, 1, 0, 1]
(1, 3) -> (0, 3) , 15 2 17 [1, 0, 1, 0]
(4, 3) -> (3, 3) , 14 3 17 [0, 1, 1, 0]
(4, 2) -> (3, 2) , 15 2 17 [0, 1, 1, 0]
(2, 2) -> (1, 2) , 15 2 17 [1, 0, 0, 0]
(0, 2) -> (0, 1) , 15 2 17 [1, 1, 1, 0]
(0, 2) -> (0, 1) , 15 2 17 [1, 1, 0, 0]
(0, 2) -> (0, 1) , 15 2 17 [0, 1, 0, 1]
(3, 2) -> (2, 2) , 16 1 17 [0, 1, 1, 0]
(1, 2) -> (1, 3) , 16 1 17 [1, 0, 0, 0]
(1, 3) -> (1, 4) , 17 inf inf [0, 0, 0, 0]
```

```
Path to collect all rewards:
(0,0) → (1,0) → (2,0) → (3,0) → (4,0) → (4,1) → (4,2) → (4,3) → (4,4) → (4,3) → (3,3) → (3,2) → (2,2) → (2,1) → (2,2) → (1,2) → (1,3) → (1,4)
Nodes explored: 198
```